UOW MALAYSIA

PART OF THE UNIVERSITY
OF WOLLONGONG AUSTRALIA
GLOBAL NETWORK

School of Engineering, Computing & Built Environment

School of Computing and Creative Media

Bachelor of Computer Science (Hons)
/ Bachelor of Software Engineering (Hons)

Advanced Programming
XBAU2014N

Prepared by Bridget Merliza a/p Paul Christopher Archibald
September 2026 Semester

SCHOOL OF COMPUTING & CREATIVE MEDIA

Event Registration and Ticket Management

Course Title : ADVANCED PROGRAMMING
Course Code : XBAU2014N
Course Lecturer : Bridget Archibald (bridget.p@uow.edu.my)
: Tan Phit Huan (phithuan.tan@uow.edu.my)

BRIEF

CLO Alignment & Problem Statement

- SLO2: Design and implement object-orieted programming solutions to problems uing C++ programming language
- SLO4: Incorporate advanced features in C++ to achieve efficient implementations

The objective of of this assignment is to implement a C++ application, Event Registration and Ticket Management System, using object-oriented programming principles. The system should demonstrate appropriate object-oriented design through classes, inheritance, composition, encapsulation, and polymorphism.

The solution should also incorporate advanced C++ features to improve program quality, maintainability, and efficiency. To align with modern digital transformation initiatives, and integrate with the UN Sustainable Development Goal (SDG 9: Industry, Innovation and Infrastructure), the application should support the management of registration workshops, seminars and competitions through file-based data storage and processing.

REQUIREMENTS :

Part A :Object-Oriented Design and Implementation - Aligned with SLO2

Implement a C++ application to simulate the management of events and participant Registrations. The system shall manage information about events, participants, registrations. The application must be developed using an object oriented approach and should model real-world relationships among different entities. Students are required to design and implement. Appropriate classes to support the following functionalities.

1. **Event management:** The system shall:
    a. Load event information from a text file.
    b. Display available events.
    c. Store event details such as event ID, event name, event type, registration fee. Examples of events include workshop, seminar, competition.
2. **Participant Management:** The system shall:
    a. Load participant information from a text file.
    b. Display participant records.
    c. Store participant details such as participant ID, participant name.
3. **Registration management:** The system shall:
    a. Load registration records from a text file.
    b. Register participants for events.
    c. Cancel registrations.
    d. Display registration information.
    e. Prevent duplicate registrations. A participant may register for multiple events.
4. **Reporting functions:** The system shall generate reports such as:
    a. List of participants registered for a particular event.
    b. Registration summary for a participant.
    c. Total number of registrations for each event.
    d. Total registration fees collected for each event.
5. **Object oriented design requirements.** Students are expected to demonstrate.
    a. Classes and objects.
    b. Encapsulation. Information hiding.

c. Class relationships such as Composition. Inheritance etc. Note that not all relationships need to be used.
d. UML class diagram.

- **Inputs in Event.txt, Participant.txt and Registration.txt**
  - *Event.txt*
    - *Format:*
      eventId;eventName;eventType;registrationFee
    - *Example:*
      E001;C++ Workshop;Workshop;150
      E002;AI Seminar;Seminar;100
  - *Participant.txt*
    - *Format:*
      participantId;participantName
    - *Example:*
      P001;Ali Hassan
      P002;Siti Nurhaliza
  - *Registration.txt*
    - *Format:*
      participantId;eventId
    - *Example:*
      P001;E001
      P002;E002

Part B : Advanced C++ Programming Requirements - Aligned with SLO4

Students are expected to demonstrate proficiency beyond basic object oriented programming constructs by incorporating appropriate advanced and modern C++ features into their solutions. Marks will be awarded for the effective and meaningful use of advanced programming features that improve the quality and robustness of the system. Examples include

- ✓ **Runtime Polymorphism**. Implement virtual functions and demonstrate runtime polymorphism through base class pointers or references.
- ✓ **Construct Overloading**. Provide overloaded constructors where appropriate.
- ✓ **Operator overloading**. Implement at least one meaningful operator overloading function. Examples may include registration comparison, fee calculation, participant statistics.
- ✓ **Friend functions.** Implement at least one meaningful friend function.
- ✓ **Exception handling**. Use exception handling to manage situations such as invalid event ID, invalid participant ID, duplicate registration, file processing errors.
- ✓ **File processing**. The application shall
    - read information from the provided text files.
    - Save any modification modifications back to files when required.
- ✓ **Documentation and Referencing**. All external references used must:
    - Be cited within the source code.
    - Be included in the README file
    - Clearly explain how the resource was used during development.

    Unreferenced use of external materials may be treated as academic misconduct.

NOTE

- ✓ Do not modify the provided file structures.
- ✓ The system must be implemented using multiple source and header files.
- ✓ Use appropriate access specifiers.
- ✓ Dynamic memory allocation may be used where appropriate.
- ✓ Include suitable comments, meaningful identifiers, and proper coding formatting.
- ✓ A UML class diagram must be included in the documentation.
- ✓ Academic misconduct will result in a 0 mark and be reported to the university.

SUBMISSION REQUIREMENTS

- ✓ Create a ZIP file named campus_studentId_ass2.zip (e.g. **UGM12321_ass2.zip** or **PGM12322_ass2.zip**) and submit via **MsTeams**.
- ✓ The ZIP file must contain: **driver.cpp, header.h or header.hpp, header.cpp,** a **README file, the classDiagram.jpg** the **Gen AI Declaration**
- ✓ The README must describe
  - *System overview*
  - *Role of each source file*
  - *Compilation and execution instructions*
  - *Design designs*
  - *Reference list*

ASSESSMENT CRITERIA

Award:

50 marks for Implementation Object-Oriented Programming

50 marks for Advanced Modern C++ Requirement

DUE DATE:

13th November 2025 (Friday)

Advanced Programming (September 2026) **ASSESSMENT 2 RUBRICS (20%)**

Name : _____ **(Student id: _____)**

LEARNING OUTCOME	Fail (0-49)	3rd Class (50-59)	2nd Lower Class (60-69)	2nd Upper Class (70-79)	1st Class (80-100)	Marks (100%)	Weightage	Actual Marks
SLO2 **Object-Oriented Design and Implementation**	Program does not satisfy the assignment requirements. Classes are poorly designed or missing. Little or no evidence of encapsulation, inheritance, composition, or object interaction. UML diagram is absent or incorrect.	Program partially satisfies requirements with several design and implementation issues. Basic classes are present but object-oriented principles are applied inconsistently. UML is incomplete.	Program satisfies most requirements with appropriate use of classes, objects, encapsulation, inheritance, and composition. UML diagram represents most class relationships correctly. Minor implementation issues may exist.	Program satisfies all major requirements with good object-oriented design. Classes interact effectively and demonstrate meaningful inheritance, composition, encapsulation, and abstraction. UML diagram is clear and accurate.	Program fully satisfies all requirements and demonstrates excellent object-oriented design. The solution shows well-structured classes, effective use of inheritance and composition, strong encapsulation, clear abstraction, accurate UML design, and high-quality implementation.		**0.50**	
SLO4 **Advanced Modern C++ Features, Documentation & Referencing**	Little or no evidence of advanced C++ features. Program functionality is incomplete or incorrect. README is missing or incomplete. References are absent.	Limited use of advanced C++ features with minimal contribution to solution quality. Program functionality is partially implemented. README and references are incomplete.	Appropriate use of some advanced C++ features such as polymorphism, constructor overloading, operator overloading, friend functions, file handling, or exception handling. README adequately describes the solution and includes basic references.	Effective use of several advanced C++ features that improve maintainability, efficiency, and program quality. Program functionality is complete with minor issues. README is clear, complete, and references are properly cited in both source code and README.	Excellent and meaningful integration of advanced C++ features including runtime polymorphism, operator overloading, friend functions, constructor overloading, file processing, and exception handling. Program functionality is robust and reliable. README is comprehensive, professionally presented, and all external sources are consistently cited within the code and README with clear explanations of their use.		**0.50**	